

RAPPORT DE PROJET

ESP32 Camera Monitor

*Systeme de videosurveillance sans fil ESP32
& application de supervision C++/GTK avec detection d'objets (YOLO)*

Kelly KASSIN

Projet personnel — 2026

1. Introduction et contexte du projet

Ce projet consiste en la conception complete, « from scratch » et sans framework preexistant, d'un systeme de videosurveillance sans fil. Il repose sur deux briques logicielles distinctes mais complementaires : un firmware embarque tournant sur un microcontroleur ESP32 equipe d'une camera, et une application de supervision desktop developpee en C++ avec l'interface graphique GTK4. L'ensemble communique via le reseau Wi-Fi et le protocole HTTP, et integre un module d'intelligence artificielle (YOLO) pour la detection d'objets et de personnes en temps reel.

L'objectif pedagogique et technique du projet etait de couvrir l'integralite d'une chaine de developpement embarquee moderne : configuration bas niveau d'un microcontroleur, communication reseau, programmation concurrente (multithreading), interface graphique, et integration d'un modele d'intelligence artificielle execute localement, sans dependance a un service cloud.

1.1 Objectifs fonctionnels

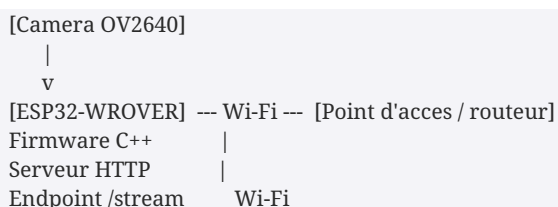
- **Acquisition video** : capturer un flux video en continu depuis une camera embarquee sur ESP32.
- **Transmission sans fil** : diffuser ce flux via un reseau Wi-Fi, sans cablage entre la camera et le poste de supervision.
- **Visualisation en temps reel** : afficher le flux recu dans une application desktop, avec une latence minimale.
- **Capture et enregistrement** : permettre la prise de photos instantanees et l'enregistrement de sequences video au format .mp4.
- **Detection intelligente** : reconnaitre automatiquement des objets et des personnes dans le flux, avec un comptage optionnel.
- **Interface complete** : proposer une experience utilisateur soignee (galerie de captures, reglages, menu, gestion de dossiers).

2. Architecture generale du systeme

Le systeme repose sur une architecture client-serveur repartie sur deux machines physiques distinctes : le microcontroleur ESP32, qui joue le role de serveur embarque, et le poste de travail (laptop), qui execute l'application cliente de supervision.

2.1 Schema de principe

Flux general de communication :



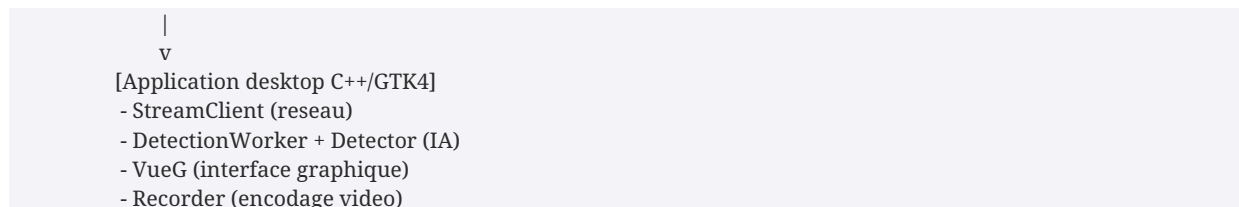


Figure 1 — Vue d'ensemble de la communication entre le firmware et l'application

2.2 Architecture multithread de l'application desktop

Un point central de la conception logicielle est l'utilisation de trois threads d'exécution indépendants au sein de l'application desktop, afin de garantir que l'interface graphique ne soit jamais bloquée, quelle que soit la charge du réseau ou du calcul d'inférence.

Thread	Classe responsable	Role
Reseau	StreamClient	Reçoit en continu le flux MJPEG via libcurl et extrait les frames JPEG individuelles du flux multipart.
Detection IA	DetectionWorker	Recupere periodiquement le dernier frame disponible et execute l'inférence YOLO via la classe Detector.
Interface (UI)	VueG	Thread principal GTK : affichage video, dessin des detections, gestion des controles utilisateur.

La communication entre ces threads est assurée de manière thread-safe grâce à deux mécanismes combinés : des verrous (`std::mutex`) protégeant les données partagées (dernier frame reçu, dernières détections calculées), et des signaux `Glib::Dispatcher` permettant à un thread de travail de notifier le thread d'interface qu'une nouvelle donnée est disponible, sans jamais appeler directement de fonction GTK depuis un thread autre que le thread principal.

2.3 Organisation des fichiers du projet

```

Firmware/                (Firmware ESP32 - PlatformIO)
platformio.ini
include/camera_pins.h
src/main.cpp

Esp_monitor_gtk/         (Application desktop - Makefile)
Makefile
main.cpp
include/
  StreamClient.hpp
  Detector.hpp
  DetectionWorker.hpp
  Recorder.hpp
  VueG.hpp
src/
  StreamClient.cpp
  Detector.cpp
  DetectionWorker.cpp
  Recorder.cpp
  VueG.cpp
models/yolo26n.onnx
  
```

3. Firmware embarque (ESP32)

Le firmware est developpe en C++ avec le framework Arduino, au sein de l'environnement PlatformIO. Il assure trois fonctions essentielles : la configuration et le pilotage de la camera, la connexion au reseau Wi-Fi, et l'exposition d'un serveur HTTP diffusant le flux video.

3.1 Configuration du projet PlatformIO

```
[env:freenove_esp32_wrover]
platform = espressif32
board = freenove_esp32_wrover
framework = arduino
monitor_speed = 115200
```

platformio.ini — configuration de la cible de compilation

3.2 Configuration des broches de la camera

Les broches (GPIO) reliant le module camera OV2640 au microcontroleur sont definies dans un fichier d'en-tete dedie, conditionne par le modele de carte utilise (ici, le kit WROVER).

```
#if defined(CAMERA_MODEL_WROVER_KIT)
#define PWDN_GPIO_NUM -1
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 21
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 19
#define Y4_GPIO_NUM 18
#define Y3_GPIO_NUM 5
#define Y2_GPIO_NUM 4
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22
#endif
```

camera_pins.h — extrait

3.3 Initialisation de la camera et du reseau

```
camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
// ... assignation des autres broches ...
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_VGA;
config.jpeg_quality = 12;
config.fb_count = 1;
config.fb_location = CAMERA_FB_IN_PSRAM;
config.grab_mode = CAMERA_GRAB_LATEST;

esp_err_t err = esp_camera_init(&config);
```

Configuration de la camera : format JPEG, resolution VGA (640x480), utilisation de la PSRAM comme buffer d'image

3.4 Serveur HTTP embarque et endpoint de streaming

Le firmware expose un serveur HTTP minimal (bibliotheque WebServer) avec un unique point d'entree, /stream, qui envoie en continu des images JPEG au format multipart. Ce format, standard pour le streaming MJPEG, permet a n'importe quel client HTTP capable de lire un flux continu (navigateur, application maison) d'afficher une video en decodant successivement chaque image separee par une chaine de delimitation (boundary).

```
void handleStream() {
  WiFiClient client = server.client();
  String response =
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: multipart/x-mixed-replace; boundary=frame\r\n\r\n";
  client.print(response);

  while (client.connected()) {
    camera_fb_t* fb = esp_camera_fb_get();
    if (!fb) { Serial.println("Camera capture failed"); break; }

    client.printf(
      "--frame\r\n"
      "Content-Type: image/jpeg\r\n"
      "Content-Length: %u\r\n\r\n",
      fb->len
    );
    client.write(fb->buf, fb->len);
    client.print("\r\n");

    esp_camera_fb_return(fb);
  }
}
```

Extrait de src/main.cpp (firmware) — gestionnaire de l'endpoint /stream

4. Application desktop : classes et responsabilites

L'application client, developpee en C++17, s'articule autour de cinq classes principales, chacune ayant une responsabilite clairement delimitée, dans un esprit proche du patron de conception Modele-Vue-Contrôleur.

Classe	Categorie	Role principal
StreamClient	Modele (reseau)	Connexion au flux HTTP de l'ESP32, extraction des frames JPEG, reconnexion.
Detector	Moteur IA	Chargement du modele YOLO et execution de l'inference sur une image.
DetectionWorker	Orchestrateur	Execution de Detector sur un thread dedie, en continu.
Recorder	Encodage video	Pilotage d'un processus ffmpeg pour produire un fichier .mp4.
VueG	Vue (interface)	Fenetre principale GTK, affichage, controles utilisateur, galerie.

4.1 StreamClient — client reseau

Cette classe est responsable de la connexion HTTP au serveur embarque sur l'ESP32. Elle utilise la bibliotheque libcurl pour recevoir un flux continu d'octets, qu'elle doit elle-meme decouper en images individuelles, puisque le protocole HTTP ne garantit aucun alignement entre les paquets reseau recus et la structure logique du flux MJPEG.

Declaration (StreamClient.hpp)

```
class StreamClient {
private:
    std::string url;
    std::string buffer;
    std::string latestFrame;
    std::mutex frameMutex;

    std::thread networkThread;
    std::atomic<bool> shouldStop{false};
    std::atomic<bool> running{false};

public:
    Glib::Dispatcher newFrameSignal;

    StreamClient(const std::string& URL);
    void connect();

    void start();
    void stop();
    void reconnect(const std::string& newUrl);

    std::string getLastFrame();
    std::string getNewUrl() const { return url; }

private:
    bool extractFrame(std::string& outFrame);
    static size_t writeCallback(char* ptr, size_t size, size_t nmemb, void* userdata);
}
```

```
};
```

Extraction d'une image depuis le flux multipart

La methode `extractFrame()` constitue le coeur du parsing du protocole. Elle recherche l'en-tete `Content-Length` pour connaitre la taille de l'image a venir, localise le debut des donnees binaires (juste apres la ligne vide separant les en-tetes du corps), verifie que suffisamment de donnees ont ete recues, puis extrait et retire du tampon les octets correspondant a une image complete.

```
bool StreamClient::extractFrame(std::string& outFrame) {
    size_t labelPos = buffer.find("Content-Length: ");
    if (labelPos == std::string::npos) return false;

    size_t numStart = labelPos + std::string("Content-Length: ").length();
    size_t numEnd = buffer.find("\r\n", numStart);
    int length = std::stoi(buffer.substr(numStart, numEnd - numStart));

    size_t headerEnd = buffer.find("\r\n\r\n", labelPos);
    if (headerEnd == std::string::npos) return false;
    size_t dataStart = headerEnd + 4;

    if (buffer.size() < dataStart + static_cast<size_t>(length))
        return false; // image pas encore entierement recue

    outFrame = buffer.substr(dataStart, length);
    buffer.erase(0, dataStart + length);
    return true;
}
```

Callback d'ecriture libcurl et notification thread-safe

```
size_t StreamClient::writeCallback(char* ptr, size_t size, size_t nmemb, void* userdata) {
    StreamClient* self = static_cast<StreamClient*>(userdata);
    if (self->shouldStop) return 0; // force curl a interrompre le transfert

    size_t totalBytes = size * nmemb;
    self->buffer.append(ptr, totalBytes);

    std::string frame;
    while (self->extractFrame(frame)) {
        {
            std::lock_guard<std::mutex> lock(self->frameMutex);
            self->latestFrame = frame;
        }
        self->newFrameSignal.emit(); // notifie le thread UI
    }
    return totalBytes;
}
```

4.2 Detector — moteur d'inference YOLO

Cette classe encapsule le modele de detection d'objets YOLO26 (format ONNX), execute via la bibliotheque ONNX Runtime. Elle prend en entree des octets JPEG bruts et retourne une liste d'objets Detection representant chacun une boite englobante, un score de confiance et un identifiant de classe.

Structure de donnees et declaration

```
struct Detection {
    float x1, y1, x2, y2; // coordonnees a l'echelle de l'image originale
    float score;
    int classId;
};

class Detector {
private:
    Ort::Env env;
    Ort::Session session{nullptr};
    static constexpr int INPUT_SIZE = 640;
    static constexpr float CONFIDENCE_THRESHOLD = 0.5f;

public:
    Detector(const std::string& modelPath);
    std::vector<Detection> detect(const std::string& jpegBytes);
    int countPersons(const std::vector<Detection>& detections);
};
```

Pretraitement : normalisation et conversion en tenseur

Le modele attend un tenseur au format [1, 3, 640, 640] avec des valeurs normalisees entre 0 et 1, organisees par plan de couleur (format planaire), alors que l'image decodee est stockee de maniere entrelacee (RGBRGBRGB...). Cette etape convertit l'un vers l'autre, en tenant compte du rowstride (nombre d'octets separant deux lignes de pixels, qui peut differer de largeur x canaux pour des raisons d'alignement memoire).

```
const uint8* pixels = resized->get_pixels();
int rowStride = resized->get_rowstride();
int channels = resized->get_n_channels();

std::vector<float> inputTensorValues(3 * INPUT_SIZE * INPUT_SIZE);

for (int y = 0; y < INPUT_SIZE; y++) {
    for (int x = 0; x < INPUT_SIZE; x++) {
        const uint8* pixel = pixels + y * rowStride + x * channels;
        float r = pixel[0] / 255.0f;
        float g = pixel[1] / 255.0f;
        float b = pixel[2] / 255.0f;

        inputTensorValues[0*INPUT_SIZE*INPUT_SIZE + y*INPUT_SIZE + x] = r;
        inputTensorValues[1*INPUT_SIZE*INPUT_SIZE + y*INPUT_SIZE + x] = g;
        inputTensorValues[2*INPUT_SIZE*INPUT_SIZE + y*INPUT_SIZE + x] = b;
    }
}
```

Inference et post-traitement

```
auto outputTensors = session.Run(Ort::RunOptions{nullptr},
    inputNames, &inputTensor, 1, outputNames, 1);

float* outputData = outputTensors[0].GetTensorMutableData<float>();
float scaleX = static_cast<float>(origWidth) / INPUT_SIZE;
```



```
float scaleY = static_cast<float>(origHeight) / INPUT_SIZE;

for (int i = 0; i < 300; i++) { // le modele produit jusqu'a 300 detections
    float score = outputData[i*6 + 4];
    if (score < CONFIDENCE_THRESHOLD) continue;

    Detection d;
    d.x1 = outputData[i*6+0] * scaleX;
    d.y1 = outputData[i*6+1] * scaleY;
    d.x2 = outputData[i*6+2] * scaleX;
    d.y2 = outputData[i*6+3] * scaleY;
    d.score = score;
    d.classId = static_cast<int>(outputData[i*6+5]);
    results.push_back(d);
}
```

Le modele utilise a ete entraine sur le jeu de donnees standard COCO (80 classes), la classe 0 correspondant a « person ». La methode countPersons() se contente de parcourir la liste des detections retenues et de compter celles dont classId vaut 0.

4.3 DetectionWorker — orchestration de la detection en arriere-plan

L'inference YOLO etant significativement plus lente que le flux video (execution sur CPU, sans acceleration materielle dediee), il est indispensable de la decoupler du thread reseau et du thread d'interface. DetectionWorker fait tourner Detector sur un thread independant, a un rythme regule, et republie ses resultats de maniere thread-safe.

```
void DetectionWorker::start() {
    if (running) return;
    running = true;

    workerThread = std::thread([this]() {
        while (running) {
            std::string frame = client.getLastFrame();

            if (!frame.empty()) {
                auto detections = detector.detect(frame);
                {
                    std::lock_guard<std::mutex> lock(detectionsMutex);
                    latestDetections = detections;
                }
                newDetectionsSignal.emit();
            }

            std::this_thread::sleep_for(std::chrono::milliseconds(200));
        }
    });
}
```

Le worker n'est actif que lorsque l'utilisateur selectionne un mode « Avec detection » dans le panneau de reglages ; il est arrete (methode stop()), avec attente propre du thread via join()) des que le mode « Sans detection » est reactive, afin de ne pas consommer inutilement des ressources CPU.

4.4 Recorder — enregistrement video

Plutot que de reimplementer un encodeur video, cette classe pilote le programme externe ffmpeg via un tube (pipe) de communication, en lui transmettant directement les images JPEG successives recues du flux.

```
void Recorder::start(const std::string& outputPath) {
    std::string command =
        "ffmpeg -y -f image2pipe -framerate 10 -i - "
        "-c:v libx264 -pix_fmt yuv420p \"\" + outputPath + \"\" "
        "> /tmp/ffmpeg_log.txt 2>&1";

    ffmpegPipe = popen(command.c_str(), "w");
    recording = true;
}

void Recorder::pushFrame(const std::string& jpegBytes) {
    if (!recording || !ffmpegPipe) return;
    fwrite(jpegBytes.data(), 1, jpegBytes.size(), ffmpegPipe);
    fflush(ffmpegPipe);
}

void Recorder::stop() {
    recording = false;
    if (ffmpegPipe) { pclose(ffmpegPipe); ffmpegPipe = nullptr; }
}
```

L'option `-f image2pipe` indique a `ffmpeg` de lire une sequence d'images independantes depuis son entree standard, a raison de 10 images par seconde, encodees ensuite au format H.264 dans un conteneur `.mp4`.

4.5 VueG — interface graphique

VueG constitue la fenetre principale de l'application (heritage de `Gtk::Window`) et assemble l'ensemble des elements visuels : affichage video, barre d'outils, galerie de captures, menu et panneau de reglages. Elle est egalement le point de convergence des signaux emis par `StreamClient` et `DetectionWorker`.

Reception d'un nouveau frame et incrustation des detections

A chaque nouveau frame reçu, la methode `onNewFrame()` decode l'image JPEG puis, si un mode de detection est actif, dessine par-dessus l'image, a l'aide de la bibliotheque de dessin vectoriel Cairo, un rectangle et une etiquette textuelle pour chaque objet detecte.

```
void VueG::onNewFrame() {
    std::string frame = client.getLastFrame();
    auto pixbuf = /* decodage JPEG via Gdk::PixbufLoader */;

    if (detectionMode != DetectionMode::None && !cachedDetections.empty()) {
        auto surface = Cairo::ImageSurface::create(
            Cairo::Surface::Format::ARGB32, width, height);
        auto cr = Cairo::Context::create(surface);
        Gdk::Cairo::set_source_pixbuf(cr, pixbuf, 0, 0);
        cr->paint();

        int personCounter = 0;
        for (const auto& d : cachedDetections) {
            cr->set_source_rgb(0, 1, 0);
            cr->rectangle(d.x1, d.y1, d.x2 - d.x1, d.y2 - d.y1);
            cr->stroke();

            std::string label = CLASS_NAMES[d.classId];
            if (detectionMode == DetectionMode::WithCount && d.classId == 0)
                label += " #" + std::to_string(++personCounter);

            cr->move_to(d.x1 + 4, d.y1 - 5);
            cr->show_text(label);
        }
        pixbuf = Gdk::Pixbuf::create(surface, 0, 0, width, height);
    }
    img.set(pixbuf);
}
```

Communication securisee entre threads : le mecanisme Glib::Dispatcher

Un point cle de l'architecture est que ni `StreamClient` ni `DetectionWorker` n'appellent jamais directement de fonction GTK. Ils se contentent d'emettre un signal (« sonnette ») via `Glib::Dispatcher::emit()`, qui garantit que le gestionnaire connecte s'execute sur le thread principal de l'interface, quel que soit le thread depuis lequel le signal a ete emis. Le thread d'interface va alors lui-meme recuperer les donnees via un accesseur protege par mutex :

```
// Dans le constructeur de VueG :
client.newFrameSignal.connect(sigc::mem_fun(*this, &VueG::onNewFrame));
detectionWorker.newDetectionsSignal.connect(
    sigc::mem_fun(*this, &VueG::onNewDetections));

void VueG::onNewDetections() {
    cachedDetections = detectionWorker.getLatestDetections();
}
```

5. Modes de detection disponibles

L'application propose trois niveaux de fonctionnement, selectionnables depuis un panneau de reglages accessible via une icone dediee superposee au flux video.

Mode	Comportement	Etat du thread de detection
Sans detection	Affichage du flux video brut, sans aucun traitement.	Inactif
Avec detection — Niveau 1	Affichage des boites de detection et du nom de chaque objet reconnu.	Actif
Avec detection — Niveau 2	Boites de detection + numerotation individuelle de chaque personne reperee + comptage.	Actif

6. Outils et technologies utilises

Categorie	Outils / bibliotheques	Usage dans le projet
Langage	C++17	Langage principal, firmware et application
Firmware embarque	PlatformIO, framework Arduino	Developpement et flashage du firmware ESP32
Materiel	Freenove ESP32-WROVER + camera OV2640	Carte microcontroleur avec camera integree et P
Reseau embarque	WiFi.h, WebServer.h	Connexion Wi-Fi et serveur HTTP cote firmware
Reseau (application)	libcurl	Client HTTP, reception du flux MJPEG en continu
Interface graphique	GTK4 / gtkmm-4.0	Fenetres, boutons, popovers, boites de dialogue
Dessin 2D	Cairo (cairomm)	Dessin des rectangles de detection et du texte inc
Intelligence artificielle	YOLO26n (.onnx), ONNX Runtime	Detection d'objets et de personnes en temps reel
Encodage video	FFmpeg (via popen)	Encodage des frames captures en fichier .mp4
Build system	Makefile (GNU Make)	Compilation et edition de liens de l'application
Systeme de fichiers	std::filesystem (C++17)	Gestion des dossiers, captures, fichiers de configuration
Concurrence	std::thread, std::mutex, std::atomic, Glib::Dispatcher	Multithreading et communication thread-safe

7. Principales difficultes rencontrees et solutions apportees

7.1 Blocage de l'interface graphique par les operations reseau

Probleme : l'appel `curl_easy_perform()` est bloquant et ne retourne jamais tant que le flux MJPEG reste ouvert. Un premier essai d'appel direct depuis le thread principal figeait entierement la fenetre.

Solution : deport de la connexion sur un thread dedie (`std::thread`), avec un mecanisme d'arret cooperatif : le callback d'ecriture verifie regulierement un indicateur atomique (`shouldStop`) et retourne une valeur d'erreur pour forcer curl a interrompre le transfert proprement.

7.2 Incompatibilite de version entre le modele ONNX et ONNX Runtime

Probleme : le modele YOLO26 exporte utilisait un opset ONNX (version 22) plus recent que celui supporte par la premiere version d'ONNX Runtime installee (1.19.2), provoquant une erreur de chargement du modele.

Solution : identification precise de la version minimale d'ONNX Runtime supportant officiellement cet opset, puis mise a jour vers la version 1.23.0, installee manuellement (hors gestionnaire de paquets systeme) pour contourner une contrainte d'espace disque sur la partition racine.

7.3 Deformation de l'image lors du redimensionnement pour l'inference

Probleme : le flux video est recu au format VGA (640x480, rectangulaire), tandis que le modele YOLO attend une entree carree de 640x640 pixels, ce qui impose un etirement de l'image.

Solution : calcul de facteurs d'echelle distincts pour les axes horizontal et vertical (`scaleX`, `scaleY`), appliques lors du post-traitement pour remettre les coordonnees des boites de detection a l'echelle exacte de l'image originale avant affichage.

7.4 Blocage silencieux lors de la reconnexion au flux

Probleme : la fonctionnalite de reinitialisation de connexion restait sans effet apparent, necessitant un redemarrage complet de l'application.

Solution : identification d'un oubli dans la methode `stop()` de `StreamClient`, qui attendait la fin du thread reseau (`join()`) sans jamais positionner l'indicateur `shouldStop` demandant explicitement son arret, provoquant un blocage indefini et silencieux de ce thread.

8. Competences developpees

8.1 Firmware et systemes embarques

- Configuration bas niveau d'un peripherique camera sur microcontroleur (GPIO, horloge, PSRAM)
- Connexion et gestion des erreurs Wi-Fi sur systeme embarque
- Ecriture d'un serveur HTTP embarque, avec diffusion de flux en continu

8.2 Reseau et protocoles

- Comprehension et implementation du protocole HTTP multipart (streaming MJPEG)
- Client HTTP en C++ (libcurl), gestion de callbacks et parsing incremental d'un flux continu

8.3 Programmation C++ et concurrence

- Programmation orientee objet moderne (C++17), separation interface/implementation
- Multithreading natif (std::thread), synchronisation par mutex et variables atomiques
- Communication asynchrone thread-safe via le patron producteur/consommateur (Glib::Dispatcher)

8.4 Interface graphique

- Construction d'interfaces complexes avec GTK4/gtkmm (Box, Overlay, Popover, MenuButton, FileDialog)
- Dessin vectoriel 2D en temps reel avec Cairo, personnalisation visuelle via CSS

8.5 Intelligence artificielle embarquee

- Integration d'un modele de detection d'objets (YOLO) via ONNX Runtime en C++ natif
- Pretraitement d'image (redimensionnement, normalisation, conversion de format) et post-traitement de sortie de modele

8.6 Methodologie et outillage

- Ecriture et maintenance d'un systeme de build manuel (Makefile) avec dependances multiples
- Debogage systematique : tests isoles avant integration complete, lecture rigoureuse des messages d'erreur
- Resolution de problemes d'environnement (compatibilite de versions, contraintes d'espace disque)

9. Conclusion

Ce projet a permis de mettre en oeuvre, de maniere integree et fonctionnelle, l'ensemble d'une chaine technique allant du firmware embarque jusqu'a l'intelligence artificielle, en passant par la communication reseau et le developpement d'une interface graphique complete. Le resultat final est un systeme de videosurveillance sans fil autonome, capable de detecter et de compter des personnes en temps reel, sans dependance a un service tiers ou a une infrastructure cloud.

Au-dela de l'aspect fonctionnel, ce projet a ete l'occasion d'acquies une comprehension approfondie de problematiques transversales aux systemes embarques et logiciels modernes : la gestion de la concurrence, la robustesse face aux erreurs reseau, et l'integration de modeles d'intelligence artificielle dans un environnement aux ressources contraintes.

Plusieurs pistes d'amelioration restent envisageables : le redimensionnement de l'image d'entree du modele avec conservation du ratio d'aspect (letterboxing), l'ajout d'un filtrage de suppression des detections redondantes (NMS), ou encore le portage d'une partie du traitement d'inference directement sur le microcontroleur ESP32 via des modeles quantifies plus legers.